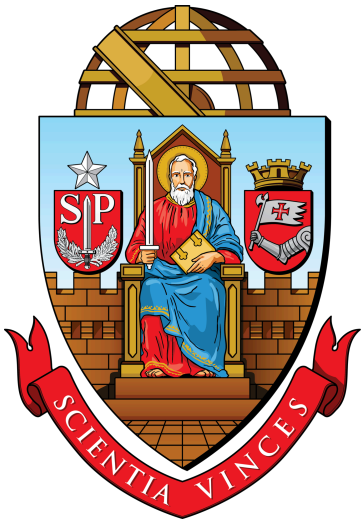




UNIVERSIDADE DE SÃO PAULO

ESCOLA POLITÉCNICA

Departamento de Engenharia de Computação e Sistemas Digitais

PCS3115 - Sistemas Digitais I

EP-02

ULA Inteira

Vinicius Castelli

NUSP: **17864509**

Professores:

Prof. Dr. Edson Satoshi Gomi

Prof. Dr. Bruno Albertini de Carvalho

São Paulo

2026

1. Introdução

O exercício prático pede uma ULA (Unidade Lógico Aritmética) completa, realizando operações de soma, subtração, multiplicação, divisão, comparação e detecção de paridade, contendo a solução, todos os testbenches e este relatório. A descrição em verilog foi realizada no modelo estrutural, sem *always* e sem marcadores aritméticos no código.

2. Representação numérica

Com 8 bits em complemento de 2, A e B vão de −128 a 127. A soma de dois valores nessa faixa vai de −256 a 254 e nem sempre cabe em 8 bits. A ULA indica isso com dois sinais de significados diferentes. O carry_out é o vai-um do bit 7 e só indica estouro quando os operandos são lidos sem sinal. O overflow indica que o resultado com sinal saiu de [−128, 127].

Em 127 + 1, o resultado é 1000_0000, que vale −128: há overflow e o carry é 0. Em −1 + 1, ou 1111_1111 + 0000_0001, o resultado é 0, o carry é 1 e não há overflow.

O módulo de um número de 8 bits vai de 0 a 128. O único valor que não cabe em 8 bits com sinal é |−128| = 128. Por isso o circuito trata o módulo como número de 8 bits sem sinal, em que 128 é 1000_0000.

3. Interface e opcodes

O módulo ula_8bit tem as portas pedidas no enunciado, com os mesmos nomes e larguras. As entradas são A[7:0], B[7:0] e opcode[1:0]. As saídas são result[7:0], carry_out, overflow, A_equal_B, A_not_equal_B, A_greater_B, A_greater_equal_B, A_less_B, A_less_equal_B, even_parity e odd_parity. A Tabela 1 mostra o efeito de cada opcode. Os sinais de comparação e de paridade não dependem do opcode.

Tabela 1: Operações da ULA

opcode	Operação	result	carry_out e overflow
00	Adição	A + B	calculados
01	Subtração	A − B	calculados
10	Multiplicação por 2	2A, em 8 bits	0
11	Divisão por 2	⌊A/2⌋	0

4. Arquitetura

Além do módulo principal, o circuito tem seis módulos. O add_sub_8bit faz a soma e a subtração, o shifter_8bit faz as duas operações por 2, e um multiplexador escolhe result conforme o opcode. Em paralelo, duas instâncias de abs_8bit calculam |A| e |B|, que entram no comparator_8bit. O parity_8bit usa só A.

A, B → [add_sub_8bit] → sum → [mux por opcode] → result, carry_out, overflow

A → [shifter_8bit] → mul2, div2 → [mux por opcode]

A, B → [abs_8bit] → |A|, |B| → [comparator_8bit] → eq, gt → seis sinais de comparação

A → [parity_8bit] → even_parity, odd_parity

4.1 Somador

O somador completo fa calcula $s = a \oplus b \oplus cin$ e $cout = a \cdot b + cin \cdot (a \oplus b)$. O adder_8bit liga oito instâncias de fa em cascata, no esquema ripple-carry: o cout de cada bit é o cin do bit seguinte, o cin do bit 0 é a entrada cin e o cout do bit 7 é a saída cout. O mesmo somador é usado no add_sub_8bit e no abs_8bit.

4.2 Soma e subtração

Em complemento de 2, A − B = A + ~B + 1. O add_sub_8bit recebe o sinal sub, ligado a opcode[0], e calcula bx = B ⊕ {8{sub}}, que liga cada bit de B a um XOR com sub. Com sub = 0, bx = B e o cin do somador é 0, o que dá A + B. Com sub = 1, bx = ~B e o cin é 1, o que dá A + ~B + 1. O mesmo somador faz as duas operações.

Há overflow quando as duas parcelas que entram no somador, A e bx, têm o mesmo sinal e o resultado s tem sinal diferente: overflow = ~(A7 ⊕ bx7)·(A7 ⊕ s7). Usar bx7 em vez de B7 faz a fórmula valer também na subtração.

Na subtração, carry_out é o vai-um de A + ~B + 1, sem inversão. Ele vale 1 quando não há empréstimo, ou seja, quando A ≥ B com os dois lidos sem sinal. O monitor confirmou que essa leitura é aceita. Em 5 − 3 = 2 o carry é 1, e em 3 − 5 = −2 o carry é 0. Em 0 − (−128) o resultado é 1000_0000 com overflow, porque +128 não cabe em 8 bits. Em −128 − 1 o resultado é 0111_1111, com overflow e carry 1.

4.3 Multiplicação e divisão por 2

As duas operações são só fiação, sem portas. A multiplicação por 2 desloca A um bit para a esquerda e põe 0 no bit 0: mul2 = {A[6:0], 0}. O bit 7 de A é descartado e, como pede o ponto 2 do enunciado, a operação não gera vai-um, e carry_out fica em 0. O overflow também fica em 0, como o monitor indicou, mesmo quando o resultado troca de sinal. É o que acontece em 64 × 2, que dá 1000_0000 (−128), e em −128 × 2, que dá 0.

A divisão por 2 desloca A um bit para a direita e repete o bit de sinal no bit 7: div2 = {A[7], A[7:1]}. Esse deslocamento aritmético preserva o sinal de A, como pede o ponto 1. Um deslocamento lógico poria 0 no bit 7 e transformaria −2 (1111_1110) em 127 (0111_1111). O deslocamento aritmético arredonda para baixo: 7/2 dá 3, −1/2 dá −1 e −3/2 dá −2. Em todos os casos, o quociente q satisfaz 2q + A0 = A, em que A0 é o bit descartado. O testbench usa essa relação para conferir a divisão.

4.4 Seleção do resultado

O sinal result sai de um multiplexador 4:1 escrito com o operador ?: encadeado, no formato visto na aula de multiplexadores. Para os opcodes 00 e 01 ele escolhe sum, para 10 escolhe mul2 e para 11 escolhe div2. Os sinais carry_out e overflow usam a mesma estrutura, com 0 nos opcodes 10 e 11, que é a forma sugerida pelo monitor.

4.5 Módulo e comparador

O ponto 4 do enunciado diz que o comparador é somente de magnitude. Interpretei isso como comparar |A| com |B|. Nessa leitura, −5 e 5 são iguais, −1 é menor que −8 e −128 é maior que 127.

O abs_8bit calcula o módulo em complemento de 2. Se o bit de sinal X7 é 0, X já é o módulo. Se é 1, o módulo é ~X + 1. Uma única expressão cobre os dois casos: o circuito faz X ⊕ {8{X7}} e soma X7 no adder_8bit, com X7 no cin. Para −128, o resultado é 1000_0000, que lido sem sinal vale 128.

O comparator_8bit recebe |A| e |B| como números sem sinal, chamados x e y. A igualdade é o AND dos XNOR bit a bit, e eq vale 1 quando todos os bits coincidem. A relação de maior é um comparador iterativo que vai do bit 0 ao bit 7, com g0 = 0 e g(i+1) = xi·~yi + ~(xi ⊕ yi)·gi. Em cada passo, x passa a ser maior se tem 1 onde y tem 0. Se os dois bits são iguais, fica a decisão tomada nos bits menos significativos. Assim, quem decide é o bit mais significativo em que x e y diferem, e gt = g8. A Tabela 2 mostra como os seis sinais saem de eq e gt.

Tabela 2: Sinais de comparação

Sinal	Expressão
A_equal_B	eq
A_not_equal_B	~eq
A_greater_B	gt
A_greater_equal_B	gt + eq

A_less_B	~(gt + eq)
A_less_equal_B	~gt

4.6 Paridade

O sinal odd_parity é o XOR dos oito bits de A e vale 1 quando A tem uma quantidade ímpar de bits em 1. O even_parity é o complemento dele. Em A = 0000_0000 não há bits em 1, e a paridade é par. Em A = 1000_0000 há um bit em 1, e a paridade é ímpar.

5. Decisões de projeto

O enunciado deixa alguns pontos em aberto. A Tabela 3 mostra como cada um foi resolvido e de onde veio a decisão.

Tabela 3: Decisões de projeto

Questão	**Decisão**	**Fonte**
carry_out na subtração	vai-um de A + ~B + 1 (1 = sem empréstimo)	Monitor
carry_out e overflow em mul/div	0	Monitor
Valores esperados fixos no testbench	aceitos	Monitor
Arredondamento da divisão	para baixo (−3/2 = −2)	Consequência do deslocamento aritmético (ponto 1)
Comparação de magnitude	|A| com |B|	Leitura do ponto 4
Paridade	calculada sobre A	Enunciado

6. Verificação

Antes da integração, cada bloco foi testado com um testbench próprio, que percorre todas

as combinações de entrada e compara a saída com o valor esperado calculado no testbench. Os testes foram tb_fa_1bit (8 casos), tb_adder_8bit (131.072), tb_add_sub_8bit (131.072), tb_shifter_8bit (256), tb_abs_8bit (256), tb_comparator_8bit (65.536) e tb_parity_8bit (256), e todos passaram.

O testbench entregue, tb_ula_8bit.v, testa a ULA completa em duas partes. A simulação foi feita no Icarus Verilog, com iverilog e vvp.

A parte 1 tem 26 vetores fixos. Cada um traz, escrito à mão, o valor esperado das 14 saídas, o que o monitor confirmou ser aceito. São seis vetores para cada uma das três primeiras operações e oito para a divisão. Eles cobrem os extremos −128, −1, 0 e 127, os overflows da soma e da subtração, o carry da subtração com e sem empréstimo, a troca de sinal na multiplicação (64 e −128) e o arredondamento da divisão (7, −1, −3 e −128). Também cobrem os pares em que a comparação por módulo difere da comparação com sinal: −5 e 5, −1 e −8, −128 e 127.

A parte 2 percorre as 2^18 = 262.144 combinações de A, B e opcode e compara todas as saídas com um modelo de referência escrito com inteiros. O modelo converte A e B para valores com sinal, subtraindo 256 quando o bit 7 é 1. Ele calcula a soma e a subtração com inteiros e marca overflow quando o resultado sai de [−128, 127]. O carry esperado é o bit 8 de A + B ou de A + (255 − B) + 1. Para a divisão, o testbench confere a relação 2q + A0 = A. O operador / do Verilog arredonda em direção a zero e daria −1 para −3/2. Os sinais de comparação esperados vêm dos módulos inteiros de A e B, e a paridade esperada vem da contagem dos bits em 1 de A. O testbench imprime no máximo os dez primeiros erros e um resumo no fim.

6.1 Injeção de defeitos

Para conferir se o testbench detecta erros, foram criadas nove cópias do circuito, cada uma com um defeito, e o tb_ula_8bit rodou em todas. As nove falharam nas duas partes. A Tabela 4 mostra quantos dos 262.144 casos da parte 2 acusaram erro em cada uma.

Tabela 4: Defeitos injetados

Defeito	Casos com erro
carry_out não zerado em mul/div	65.536
overflow não zerado em mul/div	32.768
Divisão com deslocamento lógico	32.768
Comparador com A e B, sem módulo	130.048
Paridades par e ímpar trocadas	262.144
Multiplicação e divisão trocadas no mux	130.560
carry_out da subtração invertido	65.536
Módulo sem o +1	2.040
A_less_equal_B = ~eq	132.092

O defeito do módulo sem o +1 é o que menos aparece. Sem o +1, todo negativo X vira |X| − 1, e a comparação entre dois negativos não muda. Ela só muda quando um operando é negativo, o outro não, e os módulos são iguais ou diferem de 1, o que dá 510 pares de A e B, ou 2.040 casos com os quatro opcodes. Os vetores fixos também detectaram esse defeito, porque incluem o par −5 e 5.

7. Limitações

A comparação foi feita entre |A| e |B|. Se a intenção do enunciado for comparar os bits de A e B sem tirar o módulo, basta ligar A e B direto no comparator_8bit e ajustar os valores esperados de comparação no testbench. O circuito foi verificado só em simulação. O arquivo de forma de onda gerado pela parte 2 tem cerca de 98 MB e não foi incluído na entrega, mas é gerado de novo ao rodar o testbench.

8. Conclusão

A ULA atende à interface e aos quatro pontos do enunciado. Ela passou nos 26 vetores fixos e nas 262.144 combinações do teste exaustivo, e o testbench detectou os nove defeitos injetados. A Tabela 3 reúne as decisões que o enunciado não fixa. A única que ainda depende de confirmação é a leitura de magnitude no comparador.

9. Declaração de uso de IAG

O código do EP foi totalmente feito por mim. Após finalizar e testar o EP, utilizei o Claude para: Realizar testes de menor relevância no código (A “injeção de defeitos” na seção 6.1), bem como para verificar se todo o sistema correspondia milimetricamente às instruções do enunciado e rascunhar este relatório, assim como verificar e corrigir a gramática, sintaxe e coesão dele e formatar o arquivo, depois de eu finalizar o rascunho. Todas as intervenções de IAG foram verificadas por mim e não substituíram meu próprio trabalho.

Referências Bibliográficas

Aulas do Curso.

WAKERLY, John F. Digital design: principles and practices. 5th ed. Hoboken: Pearson, 2018.